UNIFIED OCR PLATFORM

Merged System Architecture
& Technical Specification

An AI-powered OCR aggregation platform with conversational tier recommendation, RAG-grounded advisory, and live document validation on real user data.

Version 1.0 | June 2026
Confidential - Internal Use Only

1 Executive Summary

The Unified OCR Platform is an AI-powered SaaS product that helps users find and validate the right OCR solution for their document processing needs. Rather than exposing underlying model identities, the platform presents a guided experience: authenticated users upload a sample document, converse with an AI advisor that asks targeted follow-up questions, and receive a tier recommendation grounded in a proprietary RAG knowledge base. The system then automatically runs the recommended tier's OCR pipeline on the user's uploaded document and displays live results - proving value before purchase.

Core Value Proposition

- Users do not need to understand OCR technology - the agent analyzes their use case and document.
- Internal OCR model identities remain hidden; only tier names and capability descriptions are public.
- Recommendations are grounded in RAG (research papers + tier spec sheets), not generic marketing claims.
- Live demo uses the real recommended tier on the user's actual document - building trust and reducing churn.
- Within each tier, the agent automatically selects the best-fit engine from a curated pool.

Design Principle

Brain decides; body executes.

The recommendation agent (brain) analyzes the document fingerprint, conversation context, and RAG evidence to recommend a tier and select an engine. The execution layer (body) runs OCR asynchronously with cost-aware routing. Every architectural choice serves accuracy of recommendation or cost-efficient execution at scale.

2 End-to-End User Journey

The product flow is intentionally linear and authenticated. Anonymous access is not supported in the MVP - this reduces abuse, controls OCR cost, and ties every session to a billable identity.

Step 1 - Sign In

User authenticates via email/password or OAuth (Google). JWT issued. No upload or chat without auth.

Step 2 - Upload Document

User uploads exactly one case document (PDF/PNG/JPG, max 10MB). Document is stored in S3 and fingerprinted for classification.

Step 3 - AI Advisor Chat

Split-screen UI: chat panel (left) + document preview (right). Agent asks structured discovery questions about document type, language, volume, complexity, and integration needs.

Step 4 - RAG Retrieval

Agent embeds summarized user profile + document metadata and retrieves relevant chunks from Pinecone (research papers + tier spec sheets).

Step 5 - Tier Recommendation

Agent presents 1 primary + 1 alternative tier with plain-language reasoning. Structured JSON payload embedded for frontend rendering.

Step 6 - Within-Tier Engine Selection

Agent queries the model registry capability matrix to select the best engine within the recommended tier for this specific document fingerprint.

Step 7 - Auto Live Demo

System automatically runs the selected engine on the uploaded document (one run per session). Results displayed side-by-side: original vs extracted text/JSON.

Step 8 - Checkout

User selects tier and completes Stripe-powered subscription purchase.

Step 9 - Dashboard

Authenticated user accesses API key, usage stats, upload history, and programmatic OCR access.

3 System Overview

3.1 High-Level Architecture

Layer	Component	Responsibility
Presentation	Next.js Frontend (TypeScript)	Auth, upload UI, chatbot, demo viewer, checkout, dashboard
Gateway	Nginx + FastAPI	Auth, routing, rate-limiting, metering hooks, request validation
Intelligence	LLM Agent + RAG Engine	Conversation, knowledge retrieval, tier recommendation, within-tier engine selection
Orchestration	Router + Job Queue	Async job dispatch, tier-to-engine routing, verdict caching
Processing	OCR Worker Service	HuggingFace / serverless GPU inference, adapter contract per engine
Data	PostgreSQL, Pinecone, Redis, S3	Persistence, vector search, caching, file storage

3.2 Request Paths

Synchronous (Advisor Chat)

Client sends message via POST /advisor/message/ -> FastAPI validates JWT -> RAG retrieval -> LLM streams response via SSE -> recommendation JSON parsed by frontend.

Asynchronous (OCR Demo & Production Jobs)

POST creates OCRJob (status=queued) -> Celery worker pulls job -> router selects engine within tier -> inference runs on serverless GPU or HF Endpoint -> result written to S3 -> frontend polls or receives SSE notification.

4 Frontend Architecture

4.1 Technology Stack

Technology	Purpose
Next.js 14 (App Router)	SSR/SSG for SEO; React for interactive chat and demo
TypeScript	Type safety across components and API calls
Tailwind CSS + shadcn/ui	Rapid, accessible UI development
TanStack Query	Server-state management, caching, background refetch
Zustand	Client state for chat session and demo state
Stripe.js	PCI-compliant checkout
Vercel / AWS Amplify	CDN-edge hosting

4.2 Route Structure

- /login, /register: Authentication flows (required before any product interaction)
- /advisor: Primary product surface: upload zone, split chat + document preview, recommendation card, auto demo results
- /checkout: Tier confirmation and Stripe payment element
- /dashboard: API keys, usage meters, job history, subscription management
- /: Public marketing site (landing, pricing tiers by name, FAQ)

4.3 Advisor Page Layout

The /advisor page is the product centerpiece. Flow within the page: (1) Upload panel shown first if no document in session; (2) once uploaded, chat activates with document preview on the right; (3) streaming chat via EventSource/SSE; (4) recommendation card appears when agent concludes; (5) demo results panel auto-populates when OCR job completes - no extra user action.

5 Backend Architecture (FastAPI)

5.1 Project Structure

```
ocr_platform/
  app/
    main.py             <- FastAPI entry, middleware, CORS
    core/               <- Settings, security, dependencies
    accounts/           <- User model, JWT auth, OAuth
    advisor/            <- Chat sessions, LLM proxy, SSE streaming
    rag/                <- Ingestion pipeline, Pinecone client, retrieval
    ocr_engine/         <- OCR jobs, Celery tasks, engine adapters
    registry/           <- Model capability matrix, tier-engine mappings
    billing/            <- Stripe integration, webhooks, usage metering
    demo/               <- Authenticated demo endpoint (1 doc, 1 run/session)
    api/v1/             <- Versioned routers and schemas
  workers/              <- Celery worker entrypoints
  tests/
```

5.2 Technology Stack

Technology	Purpose
FastAPI	Async REST API, SSE streaming, OpenAPI docs
SQLAlchemy 2.0 + Alembic	ORM and database migrations
python-jose / PyJWT	JWT access and refresh tokens
Celery 5 + Redis 7	Async OCR jobs, RAG ingestion, rate-limit counters
PostgreSQL 15	Users, subscriptions, jobs, chat, registry
Pinecone	Vector DB for RAG retrieval
LangChain	LLM orchestration, RAG chain, prompt templates
Anthropic / OpenAI SDK	LLM API (configurable via env)
boto3	S3 storage for uploads and results
Stripe SDK	Subscriptions, webhooks, customer portal
Sentry	Error tracking and performance monitoring

6 API Endpoints

All endpoints prefixed /api/v1/. Authentication via Bearer JWT unless marked PUBLIC.

Method + Path	Auth	Description
POST /auth/register/	PUBLIC	Create account
POST /auth/login/	PUBLIC	Obtain JWT tokens
POST /auth/refresh/	PUBLIC	Rotate access token
GET /auth/me/	JWT	User profile + subscription tier
POST /advisor/session/	JWT	Start session (requires prior upload)
POST /advisor/upload/	JWT	Upload case document (1 per session)
POST /advisor/message/	JWT	Send message; SSE stream reply
GET /advisor/session/{id}/	JWT	Session state + history
POST /demo/run/	JWT	Trigger auto demo OCR (1 run/session)
GET /demo/result/{job_id}/	JWT	Poll demo OCR result
POST /billing/checkout/	JWT	Create Stripe checkout session
POST /billing/webhook/	Stripe-sig	Stripe webhook handler
GET /billing/portal/	JWT	Stripe customer portal URL
POST /ocr/jobs/	JWT	Submit production OCR job
GET /ocr/jobs/{id}/	JWT	Job status and result
GET /dashboard/usage/	JWT	Usage vs tier quota

7 AI Advisor & RAG Architecture

7.1 Conversation Phases

- GREETING: Welcome; confirm uploaded document; set expectations (a few questions).
- DISCOVERY: Document type, languages, complexity (tables, equations, handwriting), volume, integration needs.
- CLARIFICATION: One follow-up maximum if answers are ambiguous.
- RETRIEVAL: Silent RAG query based on document fingerprint + conversation summary.
- RECOMMENDATION: Primary + alternative tier with reasoning; structured JSON for frontend.
- ENGINE SELECTION: Query registry for best engine within primary tier for this document profile.
- DEMO HANDOFF: Auto-trigger OCR demo on uploaded document; no user action required.

7.2 RAG Knowledge Base (Competitive Moat)

Two document types ingested into Pinecone with distinct metadata. This dual-source design allows the agent to answer both 'why does this work technically?' (research papers) and 'what does this tier actually do?' (tier spec sheets).

Document Type	Chunking	Metadata
Research Papers (PDF)	By paragraph	source_type=research_paper, capability_tags[]
Tier Spec Sheets (MD)	By section	source_type=tier_spec, tier_id, capability_tags[]

7.3 Recommendation JSON Schema

```
{
  "recommendation": {
    "primary_tier": "pro",
    "alternative_tier": "basic",
    "primary_reasons": ["Handles multi-column PDFs", "Equation extraction"],
    "alternative_reasons": ["Lower cost if volume < 500 docs/month"],
    "selected_engine": "nougat-base",
    "demo_tier": "pro"
  }
}
```

8 Tier Packaging & Within-Tier Engine Selection

Tier names and model identities are kept separate by design. The database stores tier slugs and engine mappings internally; the frontend only renders public tier names. Each tier may map to multiple OCR engines. The agent selects the best engine within the recommended tier based on document fingerprint (type, language, layout complexity, print vs handwriting, scan quality).

8.1 Tier Structure (Starting Point)

Tier Slug	Public Name	Capability Profile
free	Starter	PDF text extraction; 50 pages/month; no API
basic	Essential	Printed text + tables; 500 pages/month; API included
pro	Professional	Equations, multi-language, handwriting; 5,000 pages/month
enterprise	Enterprise	All capabilities + custom fine-tuning; unlimited

8.2 Model Registry & Capability Matrix

The model registry is the single source of truth the agent queries. Each engine record includes: tier assignment, capability tags, cost profile, cold-start characteristics, and benchmark scores per document type. Production accuracy signals feed back into scoring over time (compounding moat).

8.3 Example Tier-to-Engine Mapping

Tier	Engines (agent picks one)	Use Case Fit
Basic	trocr-base, donut-base	Clean printed text, simple forms
Pro	nougat-base, trocr-handwritten	Scientific PDFs, equations, handwriting
Enterprise	pix2struct, docTR, passthrough APIs	Complex layouts, diagrams, SLA fallback

9 OCR Processing Layer

9.1 Engine Adapter Contract

Every OCR engine is wrapped to a common interface regardless of internal implementation:

```
input(image_or_pdf, options) -> output(text, layout, confidence, timing_ms)
```

Engines are containerized and deployable to serverless GPU (Modal/RunPod) or HuggingFace Inference Endpoints. New engines follow a standard onboarding pipeline: adapter -> containerize -> auto-benchmark -> register capabilities -> canary -> promote.

9.2 Execution Tiers & Cold-Start Strategy

Tier	Engines	Hosting	Cold Start
A - Warm	Tesseract, EasyOCR, PaddleOCR	Always-warm pool	~0.3s
B - On-demand	TrOCR, Nougat, docTR, VLMs	Serverless GPU + VRAM snapshot	~8s (snapshot)
C - Passthrough	AWS/Google/Azure OCR APIs	Vendor-managed	None

Demo jobs use the real recommended tier engine (not a cheap preview). Cost is controlled by: auth gating, one document per session, one OCR run per session, and within-tier engine selection favoring lighter engines when the document fingerprint allows.

9.3 Async Job Processing

- POST /demo/run/ creates OCRJob (status=queued), enqueues Celery task.
- Worker downloads file from S3, loads engine via adapter, runs inference, uploads result to S3.
- Frontend polls GET /demo/result/{job_id}/ or subscribes via SSE.
- Failed jobs retry up to 3 times with exponential backoff.
- Usage event emitted: tenant, engine, tier, pages, compute-seconds (cold-start tracked separately).

10 Database Schema (PostgreSQL)

Table	Key Columns	Notes
users	id, email, password_hash, created_at	Email as unique identifier
subscription_profiles	user_id, stripe_*, tier_id, quota_*	Updated on Stripe webhooks
tiers	slug, public_name, quota_limit, stripe_price_id	Admin-managed
engines	slug, tier_id, hf_endpoint, capability_tags	Multiple per tier
chat_sessions	id, user_id, document_id, recommendation_tier_id	1 doc per session
chat_messages	session_id, role, content, metadata	metadata = recommendation JSON
documents	id, user_id, s3_key, fingerprint_json	User's uploaded case doc
ocr_jobs	id, user_id, tier_id, engine_id, status, s3 keys	Demo + production jobs
knowledge_documents	title, doc_type, s3_key, capability_tags	RAG source tracking
api_keys	user_id, key_hash, is_active	SHA-256 hashed
usage_events	tenant, engine, tier, pages, compute_seconds	Billing aggregation

11 Infrastructure & Deployment

Component	Technology
Frontend	Vercel or AWS Amplify
Backend API	AWS ECS Fargate or EC2 (FastAPI + Uvicorn)
Workers	Celery on separate container pool
PostgreSQL	AWS RDS PostgreSQL 15
Redis	AWS ElastiCache Redis 7
File Storage	AWS S3 (region-aware)
Vector DB	Pinecone (serverless)
OCR Inference	HF Endpoints + Modal/RunPod serverless GPU
LLM	Anthropic Claude or OpenAI GPT-4o (configurable)
Payments	Stripe Subscriptions + Customer Portal
Monitoring	Sentry + CloudWatch + Flower (Celery)
CI/CD	GitHub Actions

11.1 Production Containers

- api: Uvicorn serving FastAPI (scales to N replicas)
- celery-worker: OCR and RAG ingestion tasks
- celery-beat: Periodic quota resets, usage reports

12 Security & Compliance

Area	Design Response
Authentication	JWT (15 min access, 7 day refresh); OAuth via Google
Authorization	Tier-based permissions; HasActiveSubscription guard
Rate Limiting	Per-user limits on advisor, upload, demo (Redis sliding window)
Demo Cost Control	1 upload + 1 OCR run per session; auth required
Data Retention	Uploaded docs: 30 days default; auto-purge configurable
PII / Compliance	Encryption at rest and in transit; region-pinned storage option
VLM Refusal Trap	Router detects ID/passport refusals; auto-reroute to compliant engine
API Keys	SHA-256 hashed; shown once on creation
Webhooks	Stripe signature verification on every event

13 Billing & Metering

Each tier maps to a Stripe Product with recurring monthly Price. POST /billing/checkout/ creates a Checkout Session. Webhooks handle checkout.session.completed, subscription updates, and payment failures. Quota enforced at gateway before OCR dispatch; decremented on job completion. Usage events aggregated asynchronously for invoicing and margin analysis.

14 MVP Scope & Build Sequence

14.1 MVP In Scope

- Auth (register, login, JWT) - required before product use
- Single document upload per advisor session
- AI advisor chat with SSE streaming
- RAG ingestion pipeline + Pinecone retrieval
- Tier recommendation with structured JSON card
- Within-tier engine selection via capability matrix
- Auto live demo on real recommended tier (1 run/session)
- 2-3 tiers with 5-8 curated engines total
- Basic Stripe checkout + webhook activation
- Dashboard: API key, usage meter, job history

14.2 Deliberately Out of Scope (Phase 2+)

- Anonymous / unauthenticated demo
- Multi-document upload per session
- Full 100-engine catalogue
- Enterprise compliance (data residency, on-prem)
- Predictive pre-warm, advanced VRAM snapshotting
- Multi-region execution

14.3 Build Sequence

Phase 1 - Foundation (Weeks 1-4)

FastAPI project, auth, PostgreSQL, S3, one baseline OCR engine, upload endpoint, Next.js scaffold with login + upload UI.

Phase 2 - AI Advisor (Weeks 5-8)

RAG pipeline, chat sessions, SSE streaming, recommendation card, tier catalogue + engine registry seeded.

Phase 3 - Live Demo (Weeks 9-10)

Within-tier engine selection, auto demo trigger, result viewer, 1 run/session enforcement.

Phase 4 - Billing & Dashboard (Weeks 11-12)

Stripe checkout, webhooks, API keys, usage metering, job history.

Phase 5 - Polish & Launch (Weeks 13-14)

Landing page, security hardening, monitoring, admin tooling for tiers and knowledge base.

15 Architecture Decision Log

Decision	Choice	Rationale
Backend framework	FastAPI	Async-native, ML ecosystem fit, SSE streaming
Frontend	Next.js + TypeScript	SSR for SEO, strong React DX
LLM provider	Claude / GPT-4o (configurable)	Strong instruction-following for structured discovery
Vector DB	Pinecone	Managed, low ops; migrate to Weaviate later if needed
Auth before product	Required	Cost control, abuse prevention, billable identity
Docs per session	1	Minimize OCR cost at our end
Demo OCR	Real recommended tier	Trust and conversion; no bait-and-switch
Engine selection	Agent picks within tier	Accuracy without exposing model catalogue
Moat	RAG + tier packaging + benchmark feedback	Proprietary knowledge + execution evidence
Async processing	Celery + Redis	OCR never blocks request cycle
Payments	Stripe Subscriptions	Customer portal eliminates custom billing UI

Document End - Unified OCR Platform Architecture v1.0

Merged from product architecture and technical architecture specifications.

All technology choices subject to revision during Phase 1 validation.